

OSSP-WEEK4-SESSION4

Task1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t child1, child2;
    int status;

    printf("Parent Process: PID = %d\n", getpid());

    child1 = fork();

    if (child1 < 0) {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 completed.\n");
        exit(10);
    }

    child2 = fork();

    if (child2 < 0) {
```

```
        perror("fork failed");
        return 1;
    }

    if (child2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 completed.\n");
        exit(20);
    }

    printf("\nParent using waitpid() for Child 1...\n");
    waitpid(child1, &status, 0);

    if (WIFEXITED(status)) {
        printf("Child 1 exited with status %d\n",
               WEXITSTATUS(status));
    }

    printf("\nParent using wait() for remaining child...\n");
    wait(&status);

    if (WIFEXITED(status)) {
        printf("Child 2 exited with status %d\n",
               WEXITSTATUS(status));
    }

    printf("\nAll child processes completed.\n");
    return 0;
}
```

Output:

```

Parent Process: PID = 478
Child 1: PID = 479

Parent using waitpid() for Child 1...
Child 2: PID = 480
Child 2 completed.
Child 1 completed.
Child 1 exited with status 10

Parent using wait() for remaining child...
Child 2 exited with status 20

All child processes completed.
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how a parent process can create multiple child processes using `fork()` and synchronize their completion using `wait()` and `waitpid()`. I understood that `wait()` waits for any child process, while `waitpid()` can wait for a specific child process. I also learned how to check the child's termination status using `WIFEXITED()` and `WEXITSTATUS()`.

Task2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Source code:

```

GNU nano 2.9.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }

    printf("Parent is sleeping without calling wait().\n");
    printf("Child becomes a zombie after termination.\n");

    sleep(20);

    printf("Parent exiting.\n");

    return 0;
}

```

Output:

```
root@LAPTOP-1C0JKKG6:~# ./practical_week4_task2
Parent PID: 515
Parent is sleeping without calling wait().
Child becomes a zombie after termination.
Child PID: 516
Child exiting...
Parent exiting.
root@LAPTOP-1C0JKKG6:~# |
```

```
root@LAPTOP-1C0JKKG6:~# ps -el | grep practical_week4_task2
root@LAPTOP-1C0JKKG6:~# ps -ef | grep defunct
root      680      679  0 03:27 pts/0    00:00:00 [practical_week4] <defunct>
root      691      628  0 03:28 pts/2    00:00:00 grep defunct
root@LAPTOP-1C0JKKG6:~#
```

Observation: The child process terminates before the parent process and remains in the process table as a zombie because the parent has not called `wait()` or `waitpid()`. The zombie process is identified by the `<defunct>` status in the process table.